Université de Carthage
Faculté des Sciences de Bizerte
Département Informatique — Classe CI2

PROJET DE FIN DE MODULE

Framework Mobile Flutter + API REST PHP

Application de Gestion des Absences

« GestAbsence »

Travail en binôme
Soutenance : 20 avril 2026

Professeur : Houssem Lahiani

Année universitaire 2025–2026

Contexte du projet

Un établissement scolaire souhaite informatiser la gestion des présences et absences de ses étudiants. Vous êtes chargés, en binôme, de concevoir et développer une solution complète composée de deux parties :

Partie 1 — Backend	Partie 2 — Application Mobile
• Base de données MySQL (XAMPP) • API REST en PHP natif • Réponses au format JSON • Gestion des 3 rôles côté serveur	• Application Flutter (Dart) • Consommation de l'API via http • 3 interfaces selon le rôle connecté • Material Design 3

Référence tutoriel PHP REST : Pour la partie PHP, suivez la playlist mise à disposition sur YouTube par votre enseignant (lien communiqué en cours).

Les trois rôles de l'application

	ADMIN	ENSEIGNANT	ÉTUDIANT
Voir liste étudiants	**Oui**	Non	Non
Ajouter/modifier étudiant	**Oui**	Non	Non
Voir liste enseignants	**Oui**	Non	Non
Ajouter/modifier enseignant	**Oui**	Non	Non
Gérer les classes	**Oui**	Non	Non
Affecter séance	**Oui**	Non	Non
Voir ses séances	Non	**Oui**	Non
Faire l'appel	Non	**Oui**	Non
Voir son profil	Non	Non	**Oui**
Voir ses absences	Non	Non	**Oui**

Partie 1 — Base de données MySQL

À créer dans phpMyAdmin (XAMPP)

Script SQL fourni

Créez une base de données nommée gest_absence et exécutez le script suivant dans phpMyAdmin :

```sql
-- ==========================================
--  Base de données : gest_absence
--  Créer dans phpMyAdmin (XAMPP)
-- ==========================================

CREATE DATABASE IF NOT EXISTS gest_absence
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE gest_absence;

-- Table des utilisateurs (commun aux 3 rôles)
CREATE TABLE utilisateurs (
  id         INT AUTO_INCREMENT PRIMARY KEY,
  nom        VARCHAR(100) NOT NULL,
  prenom     VARCHAR(100) NOT NULL,
  email      VARCHAR(150) NOT NULL UNIQUE,
  password   VARCHAR(255) NOT NULL,
  role       ENUM('admin','enseignant','etudiant') NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Table des classes
CREATE TABLE classes (
  id     INT AUTO_INCREMENT PRIMARY KEY,
  nom    VARCHAR(50) NOT NULL,
  niveau VARCHAR(50)
);

-- Table des étudiants (liée à utilisateurs + classes)
CREATE TABLE etudiants (
  id             INT AUTO_INCREMENT PRIMARY KEY,
  utilisateur_id INT NOT NULL,
  classe_id      INT NOT NULL,
  FOREIGN KEY (utilisateur_id) REFERENCES utilisateurs(id) ON DELETE CASCADE,
  FOREIGN KEY (classe_id)      REFERENCES classes(id)
);

-- Table des enseignants
CREATE TABLE enseignants (
  id             INT AUTO_INCREMENT PRIMARY KEY,
  utilisateur_id INT NOT NULL,
  specialite     VARCHAR(100),
  FOREIGN KEY (utilisateur_id) REFERENCES utilisateurs(id) ON DELETE CASCADE
);

-- Table des matières
CREATE TABLE matieres (
  id  INT AUTO_INCREMENT PRIMARY KEY,
  nom VARCHAR(100) NOT NULL
);
```

```sql
-- Table des séances
CREATE TABLE seances (
  id             INT AUTO_INCREMENT PRIMARY KEY,
  enseignant_id INT NOT NULL,
  classe_id      INT NOT NULL,
  matiere_id     INT NOT NULL,
  date_seance    DATE NOT NULL,
  heure_debut    TIME NOT NULL,
  heure_fin      TIME NOT NULL,
  FOREIGN KEY (enseignant_id) REFERENCES enseignants(id),
  FOREIGN KEY (classe_id)     REFERENCES classes(id),
  FOREIGN KEY (matiere_id)    REFERENCES matieres(id)
);

-- Table des absences
CREATE TABLE absences (
  id           INT AUTO_INCREMENT PRIMARY KEY,
  seance_id    INT NOT NULL,
  etudiant_id INT NOT NULL,
  statut       ENUM('present','absent') NOT NULL DEFAULT 'present',
  UNIQUE KEY unique_appel (seance_id, etudiant_id),
  FOREIGN KEY (seance_id)   REFERENCES seances(id),
  FOREIGN KEY (etudiant_id) REFERENCES etudiants(id)
);

-- ============================================
--  Données de test
-- ============================================
INSERT INTO utilisateurs (nom,prenom,email,password,role) VALUES
  ('Admin','Système','admin@school.tn','admin123','admin'),
  ('Ben Ali','Sami','sami@school.tn','prof123','enseignant'),
  ('Trabelsi','Amine','amine@school.tn','etu123','etudiant');

INSERT INTO classes (nom,niveau) VALUES ('CI2-A','Cycle Ingénieur 2');
INSERT INTO matieres (nom) VALUES ('Développement Mobile'),('Réseaux'),('BDD');
INSERT INTO enseignants (utilisateur_id,specialite) VALUES (2,'Informatique');
INSERT INTO etudiants  (utilisateur_id,classe_id)   VALUES (3,1);
```

> **Important :** Le mot de passe est stocké en clair pour simplifier le TP. En production, utilisez password_hash() de PHP.

Partie 1 (suite)

Structure de l'API PHP

Fichiers à créer dans htdocs/gest_absence_api/

Arborescence recommandée

```
htdocs/
└── gest_absence_api/
    ├── config/
    │   └── database.php        ← connexion MySQL
    ├── auth/
    │   └── login.php           ← POST /auth/login
    ├── admin/
    │   ├── etudiants.php       ← GET / POST / PUT
    │   ├── enseignants.php     ← GET / POST / PUT
    │   ├── classes.php         ← GET / POST
    │   └── seances.php         ← GET / POST (affecter)
    ├── enseignant/
    │   ├── seances.php         ← GET (ses séances)
    │   └── absences.php        ← POST (soumettre appel)
    └── etudiant/
        ├── profil.php          ← GET (son profil)
        └── absences.php        ← GET (ses absences)
```

Format JSON attendu : Chaque endpoint retourne toujours un objet JSON. En cas de succès : {"success":1, "data":[...]}. En cas d'erreur : {"success":0, "message":"..."}.

Partie 2 — Application Flutter — GestAbsence

À développer avec Flutter + package http

Structure du projet Flutter

```
lib/
├── main.dart                       ← MaterialApp + thème MD3
├── config/
│   └── api_config.dart             ← URL de base de l'API
├── models/
│   ├── utilisateur.dart
│   ├── etudiant.dart
│   ├── seance.dart
│   └── absence.dart
├── services/
│   └── api_service.dart            ← toutes les fonctions http
└── screens/
    ├── login_screen.dart           ← commun
    ├── admin/
    │   ├── admin_home.dart
    │   ├── etudiants_screen.dart
    │   ├── enseignants_screen.dart
    │   ├── classes_screen.dart
    │   └── seances_screen.dart
    ├── enseignant/
    │   ├── enseignant_home.dart
    │   ├── mes_seances_screen.dart
    │   └── appel_screen.dart
    └── etudiant/
        ├── etudiant_home.dart
        ├── profil_screen.dart
        └── absences_screen.dart
```

Fichier config/api_config.dart

```
// Remplacez l'IP par celle de votre machine sur le réseau local
// ou utilisez 10.0.2.2 si vous testez sur émulateur Android
const String baseUrl = 'http://10.0.2.2/gest_absence_api';

// Exemples d'URLs complètes :
// $baseUrl/auth/login.php
// $baseUrl/admin/etudiants.php
// $baseUrl/enseignant/seances.php?id=2
```

Package à ajouter dans pubspec.yaml

```
dependencies:
  flutter:
    sdk: flutter
  http: ^1.2.0              # appels API REST
  shared_preferences: ^2.2.2 # sauvegarde session utilisateur
```

Écran de connexion — comportement attendu

login_screen.dart

- Deux champs : Email et Mot de passe (obscureText)
- Bouton « Se connecter » appelle POST /auth/login.php
- En cas de succès, sauvegarde l'id et le rôle avec shared_preferences
- Redirige automatiquement selon le rôle :
  - role == 'admin' → AdminHome
  - role == 'enseignant' → EnseignantHome
  - role == 'etudiant' → EtudiantHome
- En cas d'erreur, affiche un SnackBar avec le message de l'API

Détail des interfaces par rôle

Interface ADMIN

admin_home.dart

- BottomNavigationBar ou Drawer avec 4 sections : Étudiants, Enseignants, Classes, Séances
- Chaque section affiche une liste chargée depuis l'API (FutureBuilder + ListView.builder)

etudiants_screen.dart

- Récupère GET /admin/etudiants.php → affiche nom, prénom, classe
- Bouton + → formulaire d'ajout (nom, prénom, email, mot de passe, sélection de classe)
- Tap sur un étudiant → formulaire de modification (PUT)

enseignants_screen.dart

- Même logique que etudiants_screen, avec le champ spécialité

classes_screen.dart

- Liste des classes + formulaire d'ajout (nom, niveau)

seances_screen.dart

- Liste des séances existantes (matière, classe, enseignant, date, heure)
- Formulaire d'affectation : sélectionner enseignant + classe + matière + date + heure

Interface ENSEIGNANT

mes_seances_screen.dart

- Récupère GET /enseignant/seances.php?id=X les séances de l'enseignant connecté
- Affiche : matière, classe, date, heure dans des Cards
- Bouton « Faire l'appel » sur chaque séance → navigue vers appel_screen

appel_screen.dart

- Reçoit la séance sélectionnée en paramètre du constructeur
- Charge la liste des étudiants de la classe concernée
- Affiche un CheckboxListTile par étudiant (coché = présent)
- Bouton « Valider l'appel » → POST /enseignant/absences.php avec la liste complète
- Affiche un SnackBar de confirmation après envoi réussi

Interface ÉTUDIANT

profil_screen.dart

- Récupère GET /etudiant/profil.php?id=X les informations de l'étudiant connecté
- Affiche : nom, prénom, email, classe dans une Card avec CircleAvatar

absences_screen.dart

- Récupère GET /etudiant/absences.php?id=X la liste des absences
- Affiche chaque absence : matière, date, heure, statut (présent/absent)
- Utiliser une couleur différente pour les statuts (vert = présent, rouge = absent)
- Afficher le total d'absences en haut de l'écran

Barème de notation

Partie 1 — Backend PHP + MySQL (8 points)

Critère	Pts
Base de données MySQL complète (6 tables, clés étrangères, données test)	2 pts
Endpoint login fonctionnel (POST, JSON, vérification email+password)	1 pt
Endpoints Admin : étudiants et enseignants (GET + POST + PUT)	2 pts
Endpoints Admin : classes + séances (GET + POST)	1 pt
Endpoint Enseignant : ses séances + POST appel absences	1 pt
Endpoints Étudiant : profil + liste absences	1 pt

Partie 2 — Application Flutter (12 points)

Critère	Pts
Écran de connexion fonctionnel + redirection selon le rôle	2 pts
Sauvegarde session avec shared_preferences (id + rôle)	1 pt
Interface Admin : listes + formulaires ajout/modification	3 pts
Interface Enseignant : séances + appel (CheckboxListTile)	3 pts
Interface Étudiant : profil + absences colorées	2 pts
Material Design 3, navigation propre, cycle de vie respecté	1 pt

Fonctionnalités BONUS (+4 points)

Bonus	Description
+1 pt	Suppression d'un étudiant ou enseignant (DELETE côté PHP + Flutter)
+1 pt	Barre de recherche dans les listes (filtrage local en Flutter)
+1 pt	Thème sombre Flutter (ThemeMode.dark switchable)
+1 pt	Export PDF des absences d'un étudiant (package pdf ou printing)

Calendrier et modalités de rendu

Semaine	Objectif attendu
Semaine 1	Base MySQL créée + endpoints PHP login, admin et enseignant fonctionnels
Semaine 2	Tous les endpoints PHP terminés + application Flutter complète
Semaine 3 (DS)	—
Semaine 4	Finalisation, tests, bonus éventuels
Semaine 5	Soutenance devant l'enseignant (démo live de l'application)

Livrables à remettre le jour de la soutenance

1. Le dossier complet de l'API PHP (htdocs/gest_absence_api/)
2. Le fichier SQL (gest_absence.sql) exporté depuis phpMyAdmin
3. Le projet Flutter complet (dossier lib/ + pubspec.yaml)
4. Une démonstration live sur émulateur ou téléphone réel

Déroulement de la soutenance (10 minutes par binôme)

Durée	Étape	Évalué
2 min	Présentation du projet et de la répartition du travail	*Communication*
4 min	Démonstration live des 3 rôles sur l'application	*Fonctionnalités*
2 min	Explication d'un endpoint PHP au choix de l'enseignant	*Compréhension code*
2 min	Questions techniques sur le code Flutter	*Compréhension code*

Règles importantes

- Le projet doit fonctionner en live le jour de la soutenance — un projet non fonctionnel ne sera pas évalué.
- Chaque membre du binôme doit être capable d'expliquer n'importe quelle partie du code.
- Tout plagiat entre binômes entraîne la note 0 pour les deux groupes concernés.
- L'application doit tourner sur XAMPP local ou sur un serveur accessible depuis le téléphone/émulateur.